1495 UNIVERSITY OF ABERDEEN

JC4004 – Computational Intelligence

Sessions 27 & 28: Recurrent neural networks

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)
Dr Yongchao Huang (Yongchao.huang@abdn.ac.uk)
Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)

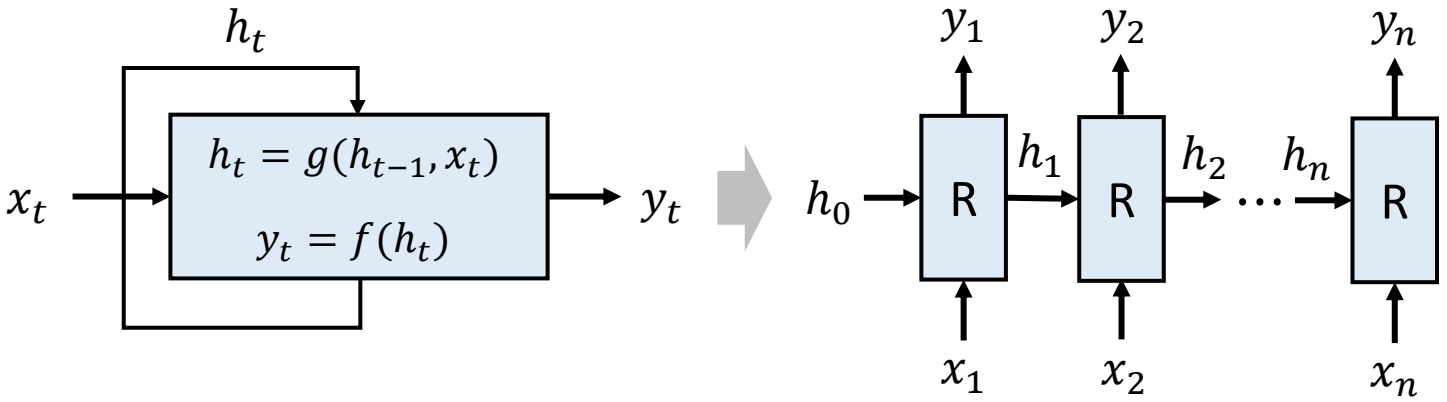
Oct-Dec 2025

What are recurrent neural networks?

- In the earlier lectures, we have introduced *feed-forward networks (FFN)* generating their predictions in a single pass
  - Appropriate for fixed length input with a single prediction result
  - However, FFNs are not optimal for processing *sequential* data: the whole sequence must be considered to make meaningful predictions
- *Recurrent neural networks (RNN)* process sequential data
  - Many practical problems include analysis of time-series data or other types of sequences (a prominent example is text in natural language)
  - RNNs can be used for sequence-to-sequence, sequence-to-one, or one-to-sequence predictions

Basic model for RNN

- In RNN, output y_t for time step t is not dependent only on the corresponding input x_t, but also on the hidden state h_t
  - Hidden state "saves" information about the previous inputs
  - Hidden state is updated for every time step, initial state is h_0



Computations in vanilla RNN

- In a basic RNN model, we can use linear transformations and proper activation functions to compute values for h_t and output y_t

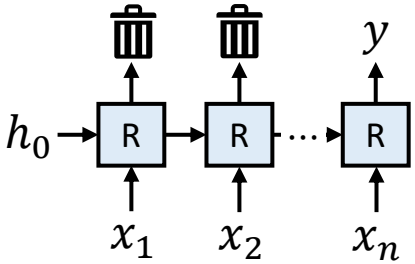
$$h_t = \varphi(W_h h_{t-1} + W_x x_t + b)$$
$$y_t = \varphi(W_y h_t)$$

  - Note that x_t , h_t , and y_t are vectors: W_h, W_x, and W_y are learnable weight matrices, and b is learnable bias; y_t is computed after h_t
  - Often, *tanh* is used as activation function in RNNs

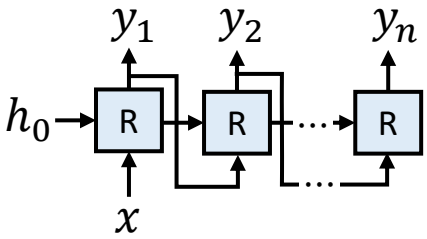
Use scenarios for RNN

- There are different usage scenarios for RNNs:
  - *Sequence-to-one:* for example, text classification or sentiment analysis
  - *One-to-sequence:* for example, image captioning
  - *Sequence-to-sequence:* for example, machine translation

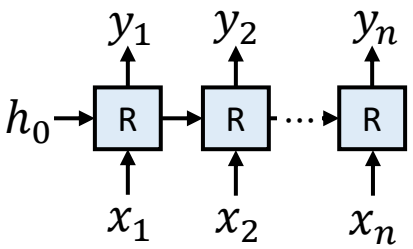
sequence-to-one



one-to-sequence

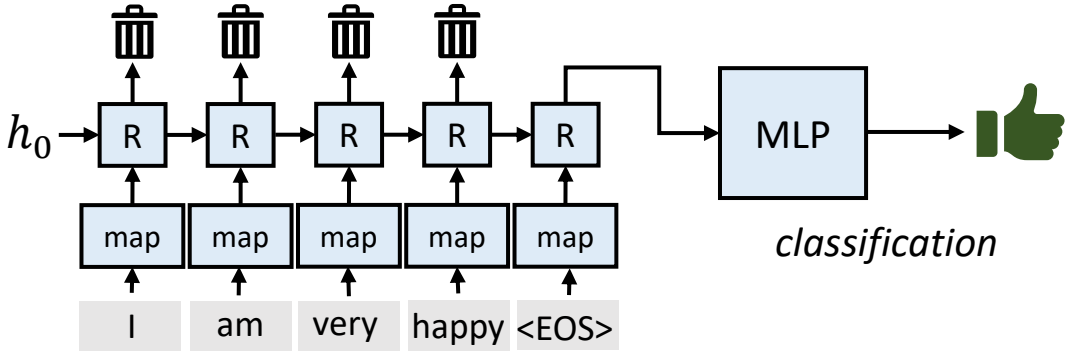


sequence-to-sequence



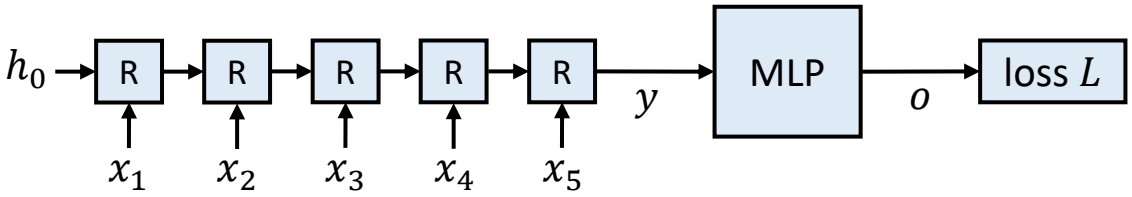
Sequence-to-one RNN

- Sequence-to-one RNNs can be used for classifying sequences
  - A common scenario is sentiment analysis of short texts
  - Text is mapped to numerical vectors (*tokens*) usually word by word; a special token can be used to denote the end of sentence, e.g., <EOS>
  - The last output value is used as input to a FFN for classification



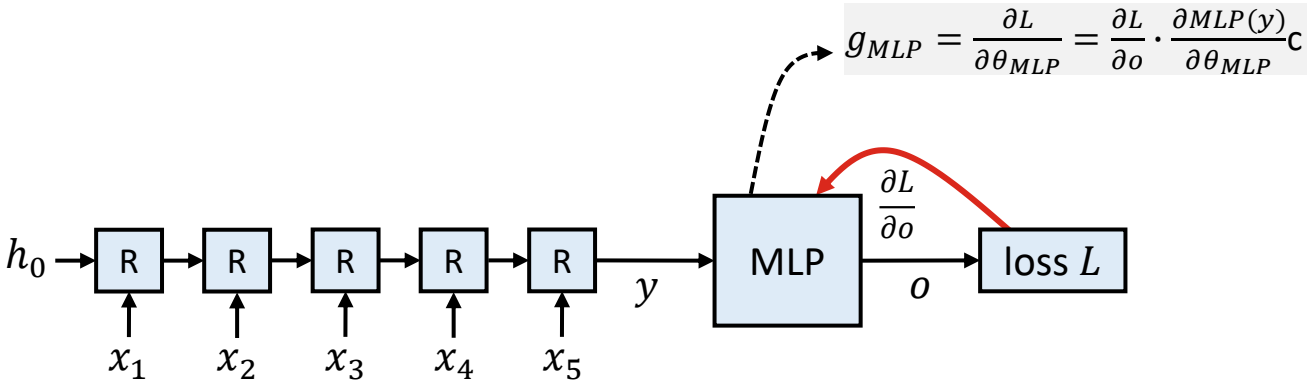
Training sequence-to-one RNN

- RNNs can be trained like FFNs through backpropagation
  - Time steps can be considered as layers with shared weights
  - The gradients for different time steps will not be the same: we must compute averages for the recurrent weight gradients for optimisation



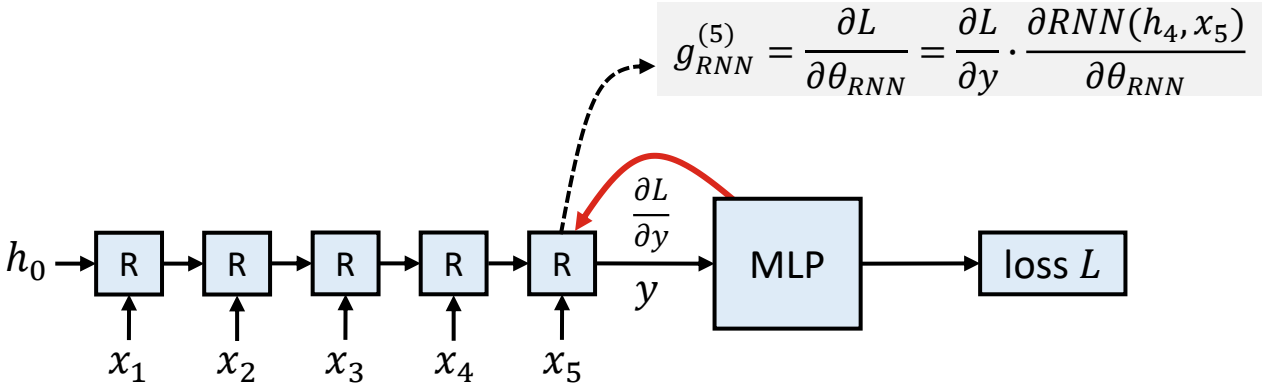
Training sequence-to-one RNN

- RNNs can be trained like FFNs through backpropagation
  - Time steps can be considered as layers with shared weights
  - The gradients for different time steps will not be the same: we must compute averages for the recurrent weight gradients for optimisation



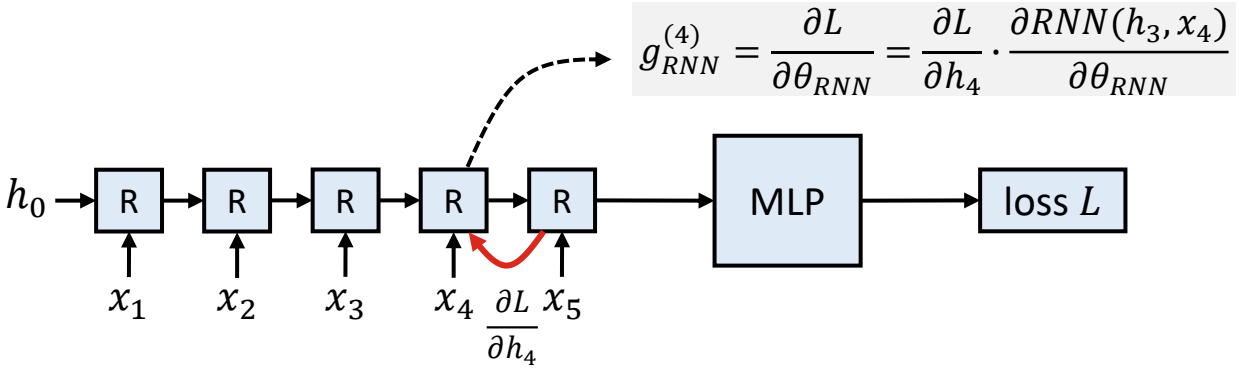
Training sequence-to-one RNN

- RNNs can be trained like FFNs through backpropagation
  - Time steps can be considered as layers with shared weights
  - The gradients for different time steps will not be the same: we must compute averages for the recurrent weight gradients for optimisation



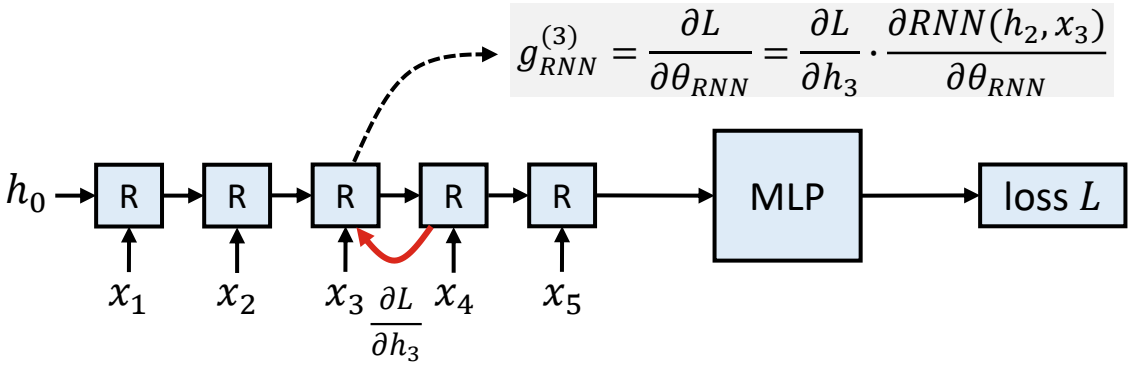
Training sequence-to-one RNN

- RNNs can be trained like FFNs through backpropagation
  - Time steps can be considered as layers with shared weights
  - The gradients for different time steps will not be the same: we must compute averages for the recurrent weight gradients for optimisation



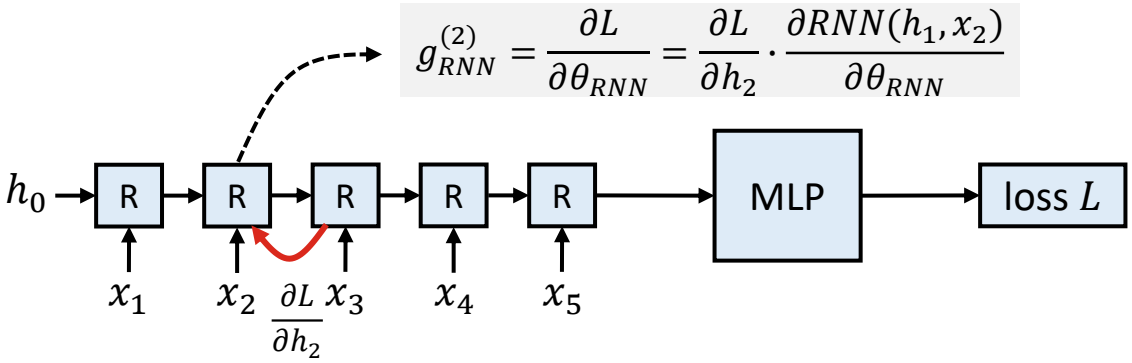
Training sequence-to-one RNN

- RNNs can be trained like FFNs through backpropagation
  - Time steps can be considered as layers with shared weights
  - The gradients for different time steps will not be the same: we must compute averages for the recurrent weight gradients for optimisation



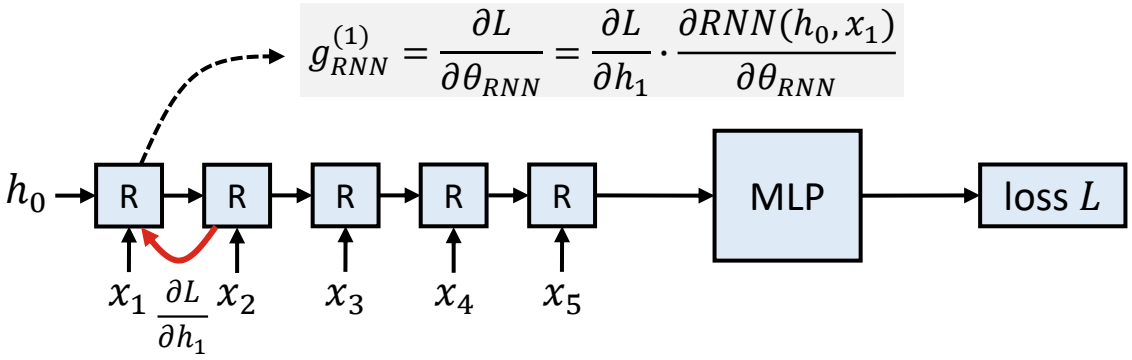
Training sequence-to-one RNN

- RNNs can be trained like FFNs through backpropagation
  - Time steps can be considered as layers with shared weights
  - The gradients for different time steps will not be the same: we must compute averages for the recurrent weight gradients for optimisation



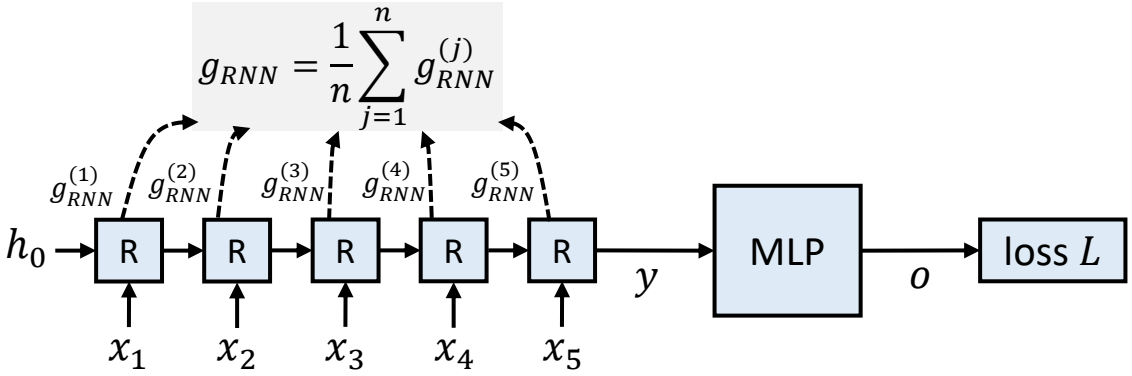
Training sequence-to-one RNN

- RNNs can be trained like FFNs through backpropagation
  - Time steps can be considered as layers with shared weights
  - The gradients for different time steps will not be the same: we must compute averages for the recurrent weight gradients for optimisation



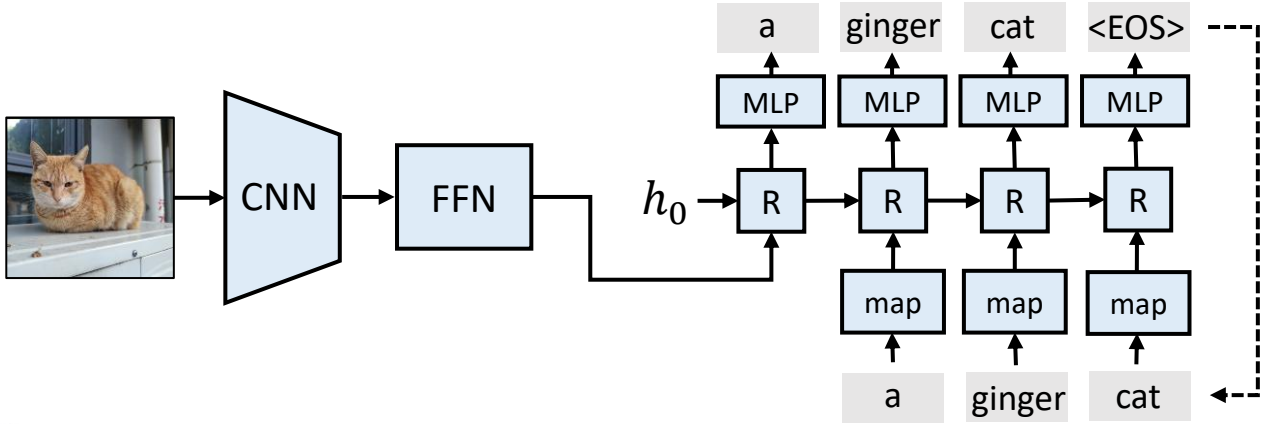
Training sequence-to-one RNN

- RNNs can be trained like FFNs through backpropagation
  - Time steps can be considered as layers with shared weights
  - The gradients for different time steps will not be the same: we must compute averages for the recurrent weight gradients for optimisation



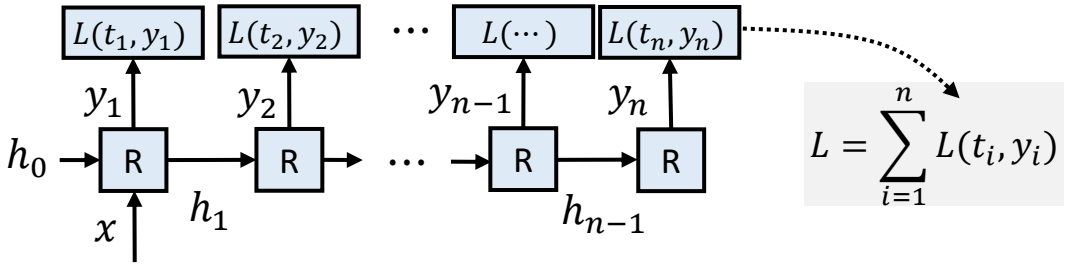
One-to-sequence RNN

- One-to-sequence RNNs can be used e.g., for generating text
  - A typical example application is image captioning: CNN is used to generate a vector representation of the image, fed to RNN
  - Generative RNNs typically use output as input to the following time steps



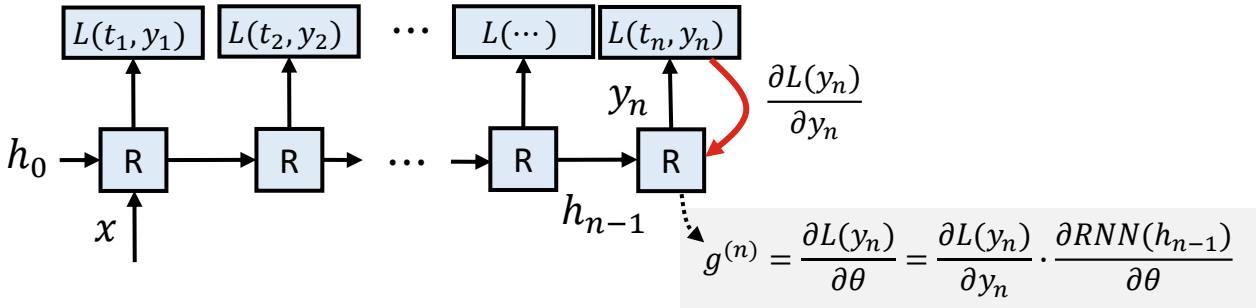
Training one-to-sequence RNN

- One-to-sequence RNNs can also be trained using backpropagation, considering time steps as layers of the network
  - However, each output contributes to the loss individually, so each time step adds an additional loss term in the total loss
  - In case of image captioning, CNN part is often fixed and needs no training



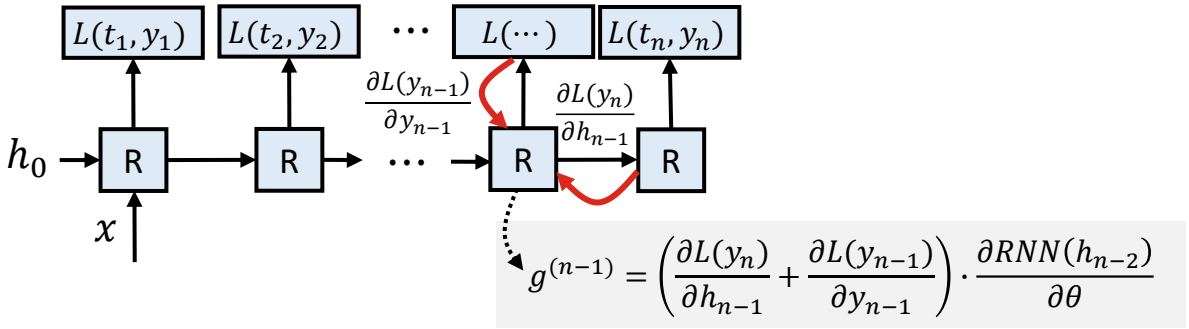
Training one-to-sequence RNN

- One-to-sequence RNNs can also be trained using backpropagation, considering time steps as layers of the network
  - However, each output contributes to the loss individually, so each time step adds an additional loss term in the total loss
  - In case of image captioning, CNN part is often fixed and needs no training



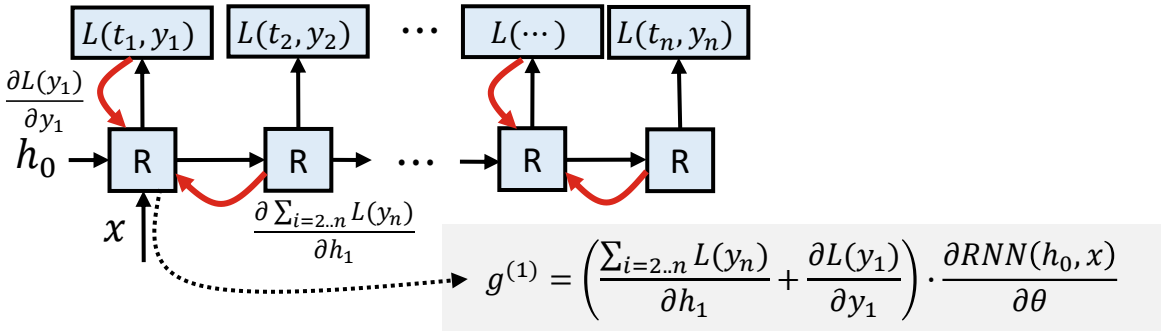
Training one-to-sequence RNN

- One-to-sequence RNNs can also be trained using backpropagation, considering time steps as layers of the network
  - However, each output contributes to the loss individually, so each time step adds an additional loss term in the total loss
  - In case of image captioning, CNN part is often fixed and needs no training



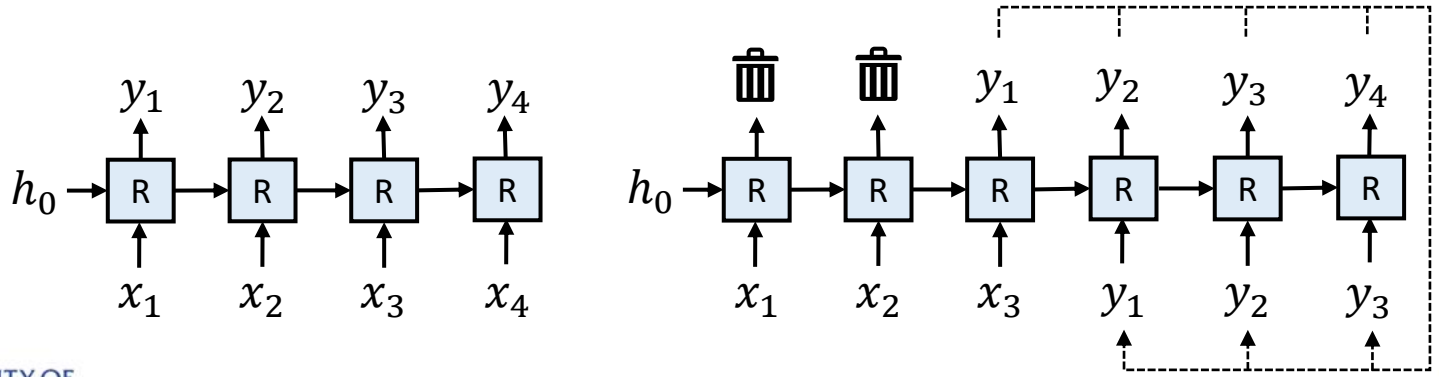
Training one-to-sequence RNN

- One-to-sequence RNNs can also be trained using backpropagation, considering time steps as layers of the network
  - However, each output contributes to the loss individually, so each time step adds an additional loss term in the total loss
  - In case of image captioning, CNN part is often fixed and needs no training



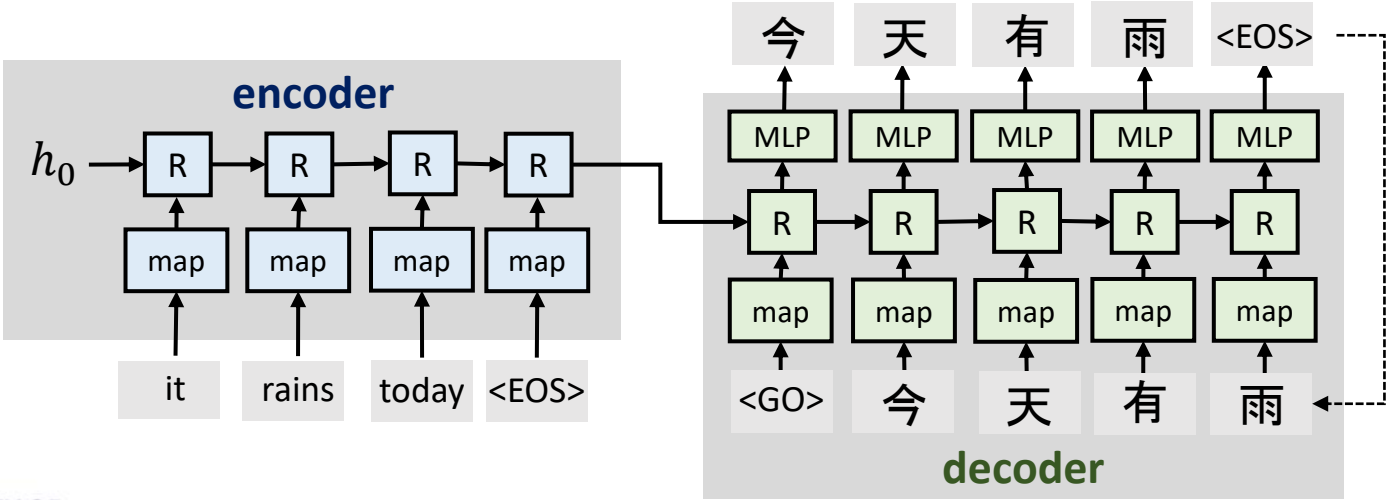
Sequence-to-sequence RNN

- Sequence-to-sequence RNNs can be implemented for synchronised output or delayed output
  - Synchronised output used for e.g., rolling prediction of time series
  - Delayed output is useful in e.g., machine translation: the system needs to receive the whole input sequence before starting to generate output



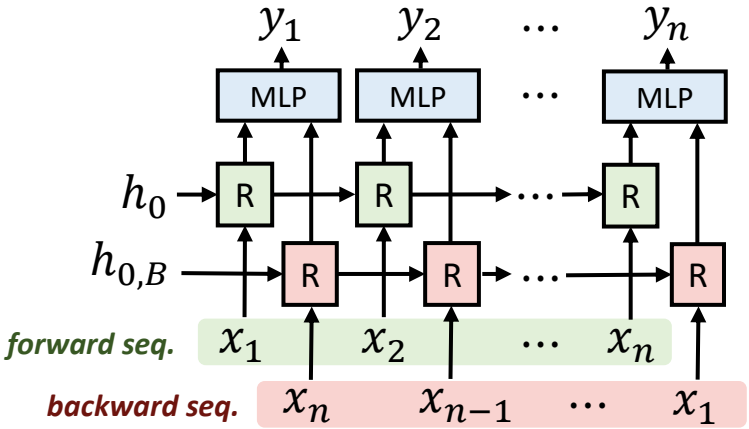
Encoder-decoder RNN

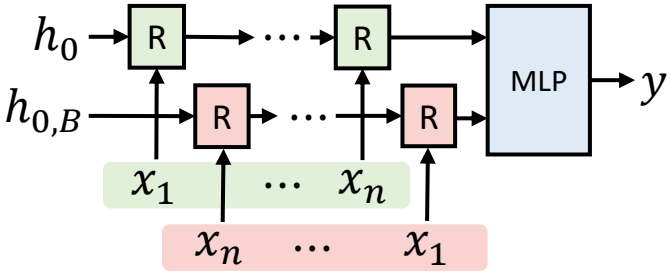
- Applications such as machine translation often use two different RNNs: encoder and decoder
  - Encoder encodes the input as hidden state, decoder generates output



Bidirectional RNN

- It can be beneficial for RNN to use also the future inputs
  - *Bidirectional RNN* generates the output using two hidden states: one computed using the original sequence, the other one using it reversed





Break: any questions?

Improving the RNNs: GRU and LSTM

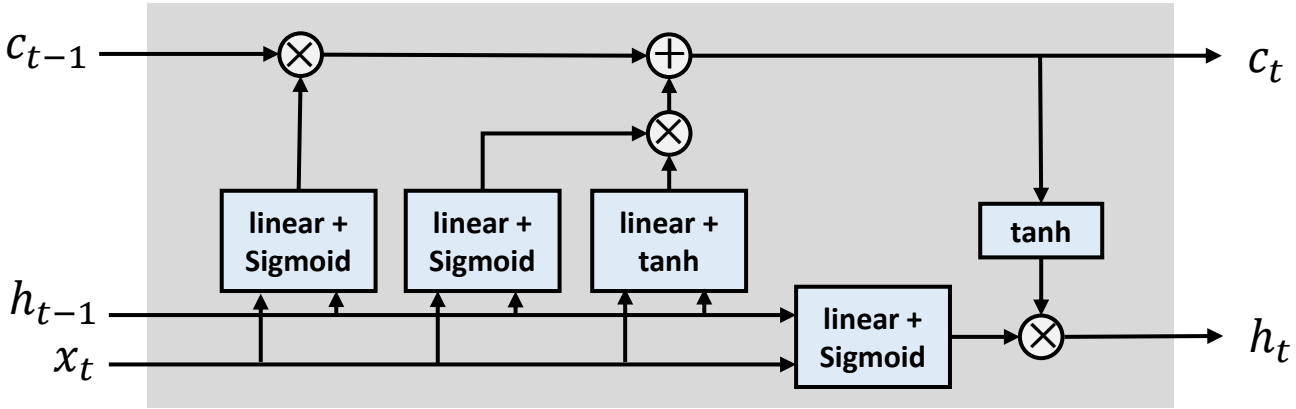
- The vanilla RNN works well on short sequences
  - However, for reliable long-term predictions, the hidden state should include information of *all* the past inputs
  - As the hidden state is updated at every time step, the information from the past gets diluted, since deep models suffer from vanishing gradient
- To improve RNN performance on longer sequence, two improved RNN layer architectures are commonly used: LSTM and GRU
  - *Long-short term memory (LSTM)* is the most common RNN variant
  - *Gated recurrent unit (GRU)* is a less complex alternative to LSTM

Long short-term memory (LSTM)

- LSTM was intended to provide a short-term memory that can last thousands of time steps, i.e., *long* short-term memory
  - Name for LSTM was inspired by psychological research on long-term and short-term memory since the early 20th century
- The intuition behind LSTM is that the networks learn when to remember and when to forget information
  - LSTM has three types of gates to control the flow of information
  - Each gate is in practice a linear layer with learnable weights and biases
  - In addition to the hidden state h_t, LSTM has also memory cell c_t

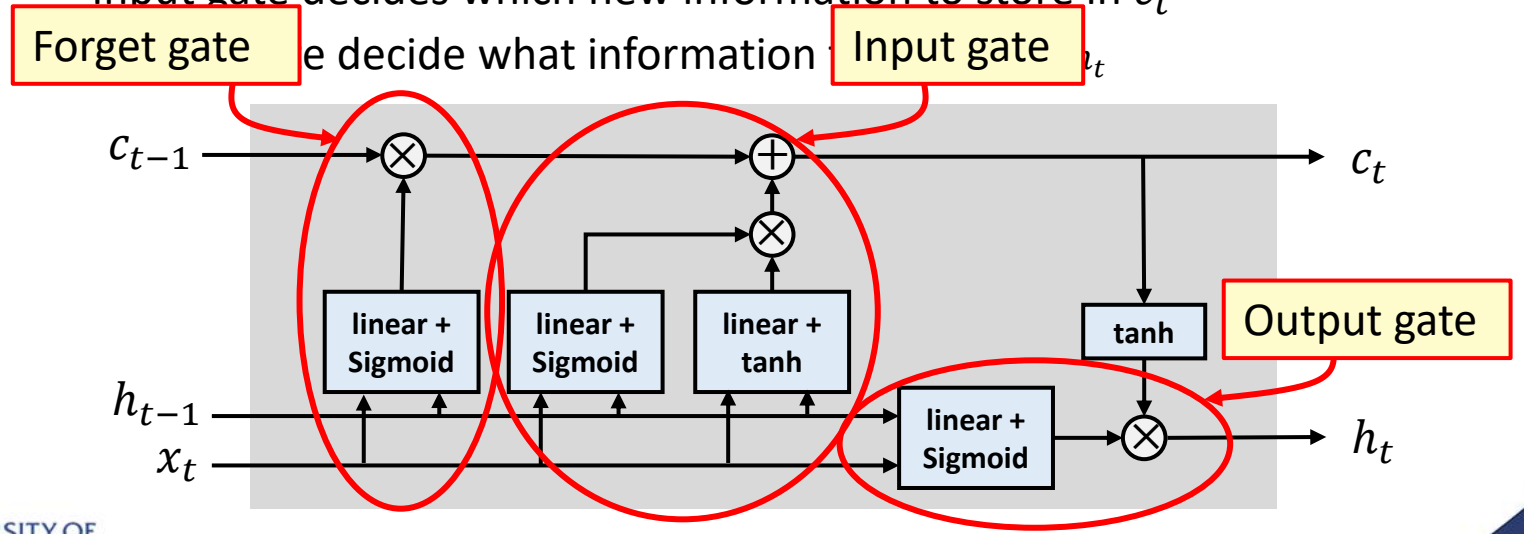
LSTM architecture

- LSTM has four gates to decide which information to forget and keep
  - Forget gate decides which information to discard from c_{t-1}
  - Input gate decides which new information to store in c_t
  - Output gate decide what information to output in h_t



LSTM architecture

- LSTM has four gates to decide which information to forget and keep
  - Forget gate decides which information to discard from c_{t-1}
  - Input gate decides which new information to store in c_t
  - e decide what information h_t



LSTM architecture

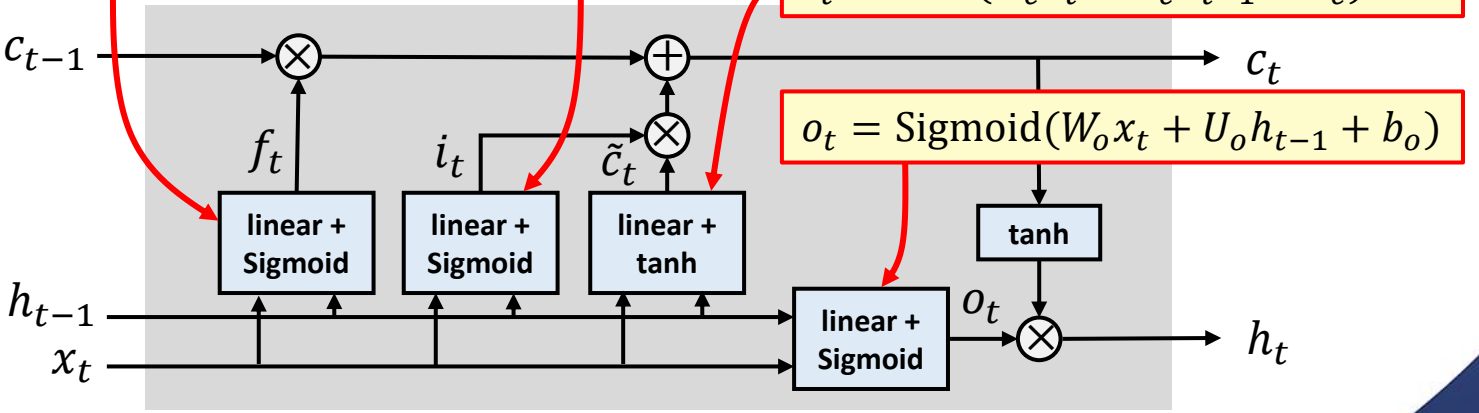
- LSTM has four gates to decide which information to forget and keep
  - Forget gate decides which information to discard from c_{t-1}
  - Output gate decide what information to

$$f_t = \text{Sigmoid}(W_f x_t + U_f h_{t-1} + b_f)$$

$$i_t = \text{Sigmoid}(W_i x_t + U_i h_{t-1} + b_i)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

$$o_t = \text{Sigmoid}(W_o x_t + U_o h_{t-1} + b_o)$$

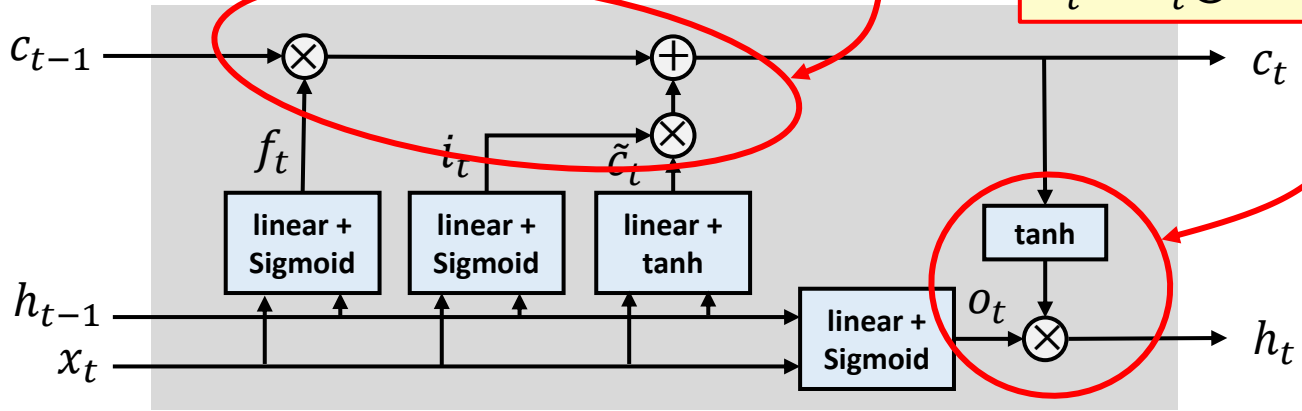


LSTM architecture

- LSTM has four gates to decide which information to forget and keep
  - Forget gate decides which informatic
  - Input gate decides which new inform
  - Output gate decide what information to output in h_t

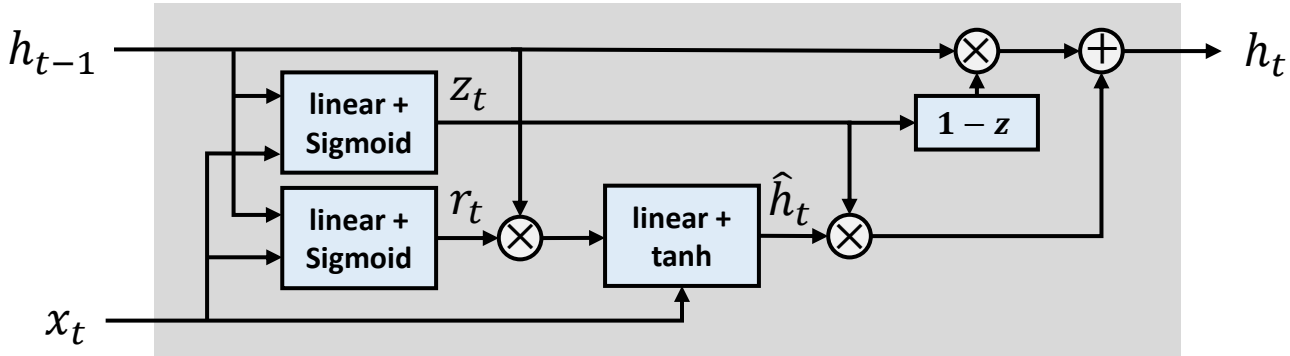
$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, $\odot$ denotes Hadamard (elementwise) product

$h_t = o_t \odot \tanh(c_t)$



Gated recurrent unit (GRU)

- GRU has been developed later than LSTM; however, GRU architecture is simpler with less gates and hidden information
  - In many tasks, GRU performs as well as LSTM
  - GRU includes a gating mechanism to input or forget certain features
  - GRU does not have an output vector: hidden state can be used as output

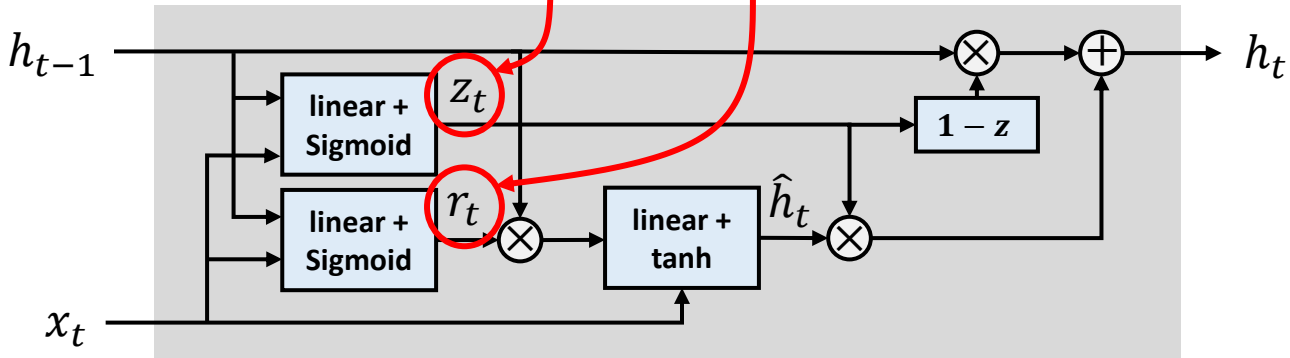


Gated recurrent unit (GRU)

- GRU has been developed later than LSTM; however, GRU architecture is simpler with less gates and hidden information

z_t is the *update gate vector* that emphasises the elements in the hidden state that should be updated

r_t is the *reset gate vector* that emphasises the elements in the hidden state that should be forgotten



Gated recurrent unit (GRU)

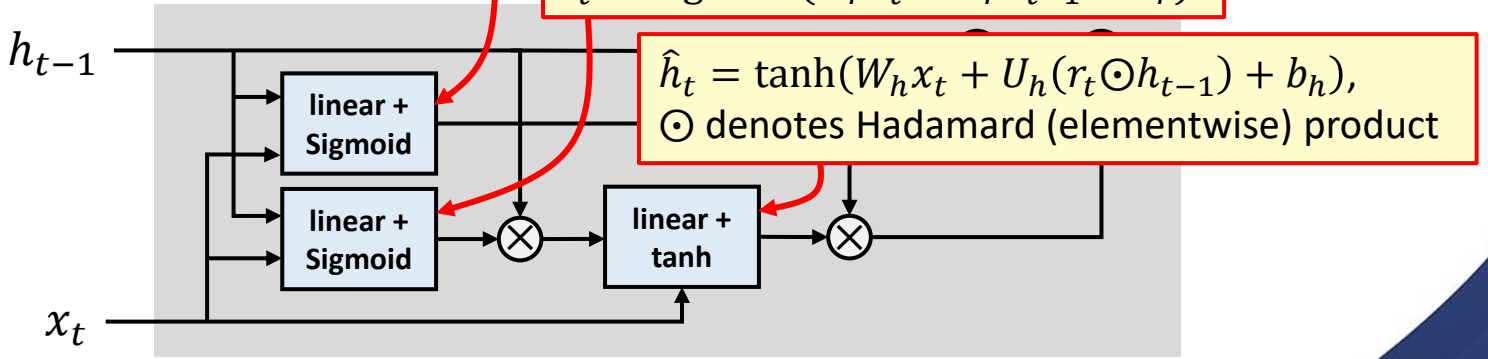
- GRU has been developed later than LSTM; however, GRU architecture is simpler with less gates and hidden information
  - In many tasks, GRU performs as well as LSTM
  - GRU includ… forget certain features
  - GRU does not have an output …s output

$$z_t = \text{Sigmoid}(W_z x_t + U_z h_{t-1} + b_z)$$

$$r_t = \text{Sigmoid}(W_r x_t + U_r h_{t-1} + b_r)$$

$$\hat{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h),$$

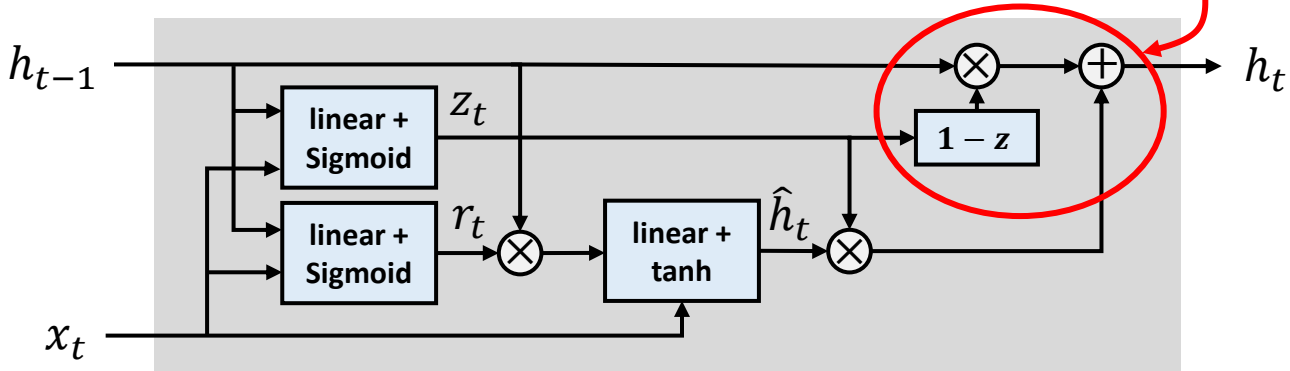
$\odot$ denotes Hadamard (elementwise) product



Gated recurrent unit (GRU)

- GRU has been developed later than LSTM; however, GRU architecture is simpler with less gates and hidden information
  - In many tasks, GRU performs as well as LSTM
  - GRU includes a gating mechanism to input or forget certain features
  - GRU does not have an output vector: hidd

$$h_t = (1 - z_t) \odot h_{t-1} + \hat{h}_t \odot z_t$$



Summary

- *Recurrent neural networks* are designed to process sequential data
  - RNNs are commonly used for natural language processing (NLP) tasks, since natural language is by nature a sequence of words
  - RNNs used for e.g., classifying data, time series analysis and forecasting, and sequence-to-sequence conversion (e.g., translation)
  - RNNs trained via backpropagation: time steps can be unfolded as layers
- Vanilla transformers memorise long-term dependencies poorly
  - *Long short-term memory (LSTM)* and *gated recurrent unit (GRU)* architectures are commonly used for more advanced RNN models

Questions and discussion